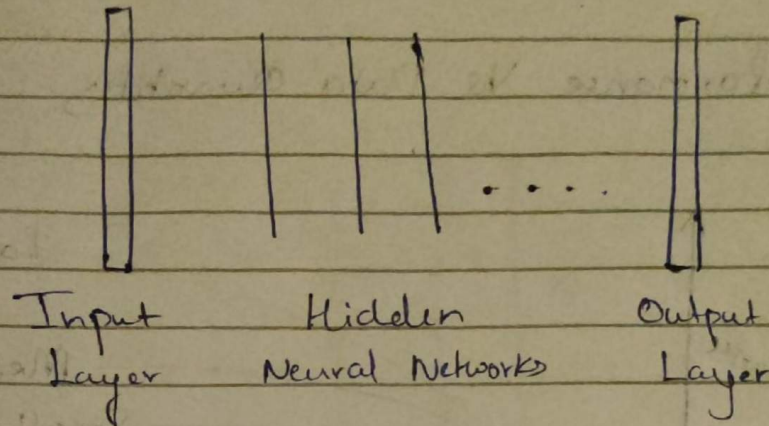


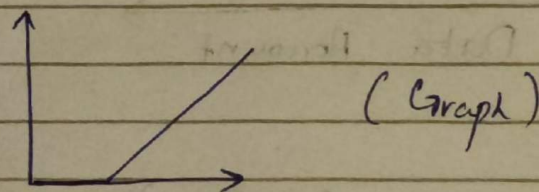
Course - 1

Neural Networks and Deep Learning

Deep Learning $\rightarrow$ Neural Networks $\rightarrow$ Neurons



ReLU $\rightarrow$ Rectified Linear Unit



Supervised Learning $\rightarrow$ [Classification & Regression]

Classification $\rightarrow$ Discrete

Regression $\rightarrow$ Continuous

Visual $\rightarrow$ CNN

Audio $\rightarrow$ RNN

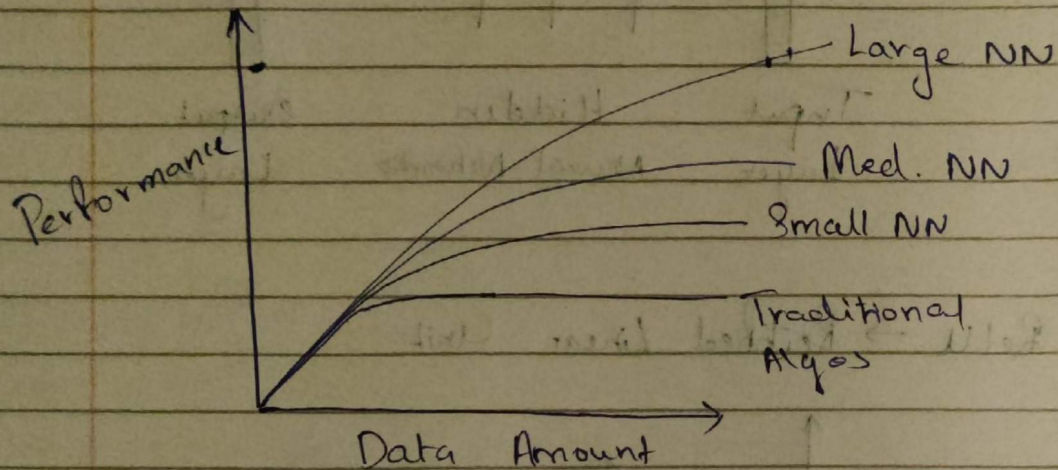
Basics $\rightarrow$ Standard NN

* Structured & Unstructured Data

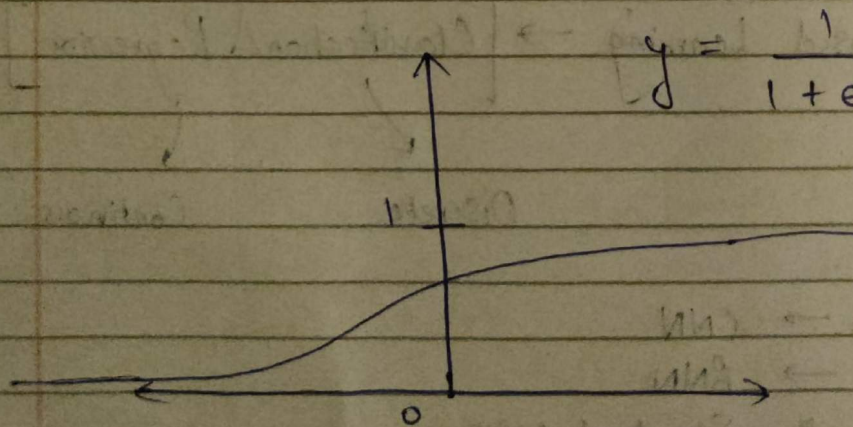
Structured $\rightarrow$ Tabular organized data

Unstructured $\rightarrow$ Unorganized data (Audio, Image)

Performance Vs Data Quantity



Sigmoid Function (0 & 1)



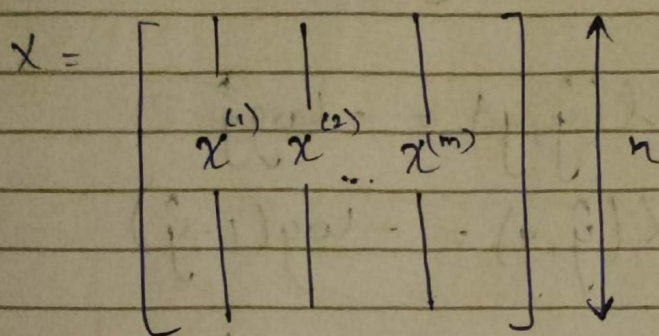
* Binary Classification (0 or 1)

$x \rightarrow$ Input

$y \rightarrow$ Output

$$x \in \mathbb{R}^n$$

$$y \in \{0, 1\}$$

$$X = \begin{bmatrix} | & | & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & | \end{bmatrix}$$


$$y = [y^1, y^2, \dots, y^m]$$

* Logistic Regression

$$x \in \mathbb{R}^n$$

$$\hat{y} = P(y=1/x)$$

Weight
Bias

} IMP

$$z = (\sum w x + b)$$

output $\rightarrow \hat{y} = \sigma(w^T x + b)$

* Cost Function

What we need $(\hat{y} \approx y)$

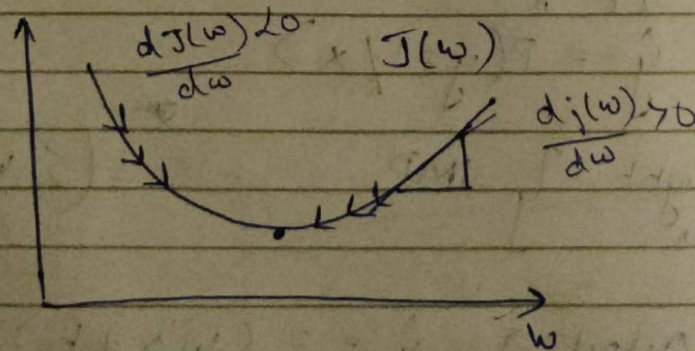
$$\text{Loss function: } \mathcal{L}(\hat{y}, y) = \frac{1}{2} (\hat{y} - y)^2 \\ = (y \log \hat{y} + (1-y) \log(1-\hat{y}))$$

$$\text{If } y=1: \mathcal{L}(\hat{y}, y) = -\log \hat{y}$$

$$\text{If } y=0: \mathcal{L}(\hat{y}, y) = -\log(1-\hat{y})$$

$$\text{Cost function: } J(w, b) = \frac{1}{M} \sum_{i=1}^M \mathcal{L}(\hat{y}^{(i)} - y^{(i)}) \\ = \frac{1}{M} \sum_{i=1}^M [y^{(i)} \log \hat{y}^{(i)} + (1-y^{(i)}) \log(1-\hat{y}^{(i)})]$$

* Gradient Descent



$$w: w = \alpha \frac{dJ(w)}{dw}$$

α = rate of learning

for $J(w, b)$;

$$w = w - \alpha \frac{dJ(w, b)}{dw}$$

$$b = b - \alpha \frac{dJ(w, b)}{db}$$

→ REPETITION

* Computation Graph → Just breakdown function into variables and express it simplified

* Computing Derivatives

Basically, just finding derivative of a var. wrt to another variable, 'just pump up each value by (.001) and calc. difference.

* How do we adjust the values of w & b

$$\begin{array}{l} x_1 \\ w_1 \\ b \\ x_2 \\ w_2 \end{array} \rightarrow [z = w_1 x_1 + w_2 x_2 + b] \rightarrow [a = \sigma(z)] \rightarrow [L(a, y)]$$
$$d_2 = \frac{dL(a, y)}{dz} \quad \frac{dL(a, y)}{da} = da$$

$$dw_1 = x_1 d_2$$

$$dw_2 = x_2 d_2$$

$$db = d_2$$

$$w_1 = w_1 - \alpha dw_1$$

$$w_2 = w_2 - \alpha dw_2$$

$$b = b - \alpha db$$

* Vectorization (repeating for loop)

$$z = w^T x + b$$

Non-vectorized

$z = 0$

for i in range(n, x):

$z += w^T x_i$

$z += b$

Vectorized

import numpy as np

$z = np.dot(w, x) + b$

Vectorizing Logistic Regression } IMP
Vectorization of gradient descent }

Some NumPy commands:

.dot()

.sum()

.reshape()

* Broadcasting

$$\text{Eg: } \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix} + 10$$

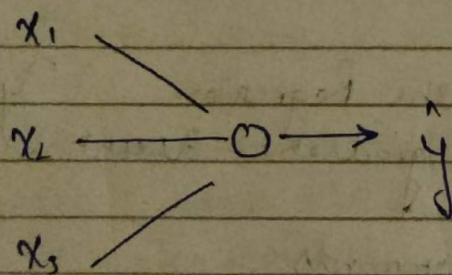
X

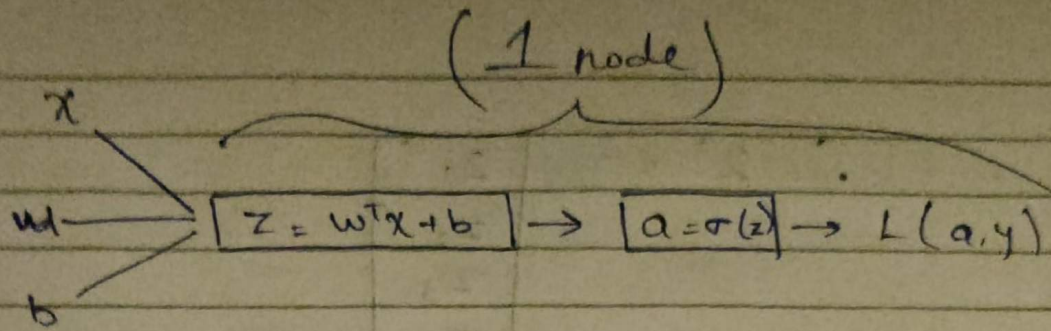
$$= X + \begin{bmatrix} 10 & 10 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

(Done By Python)

$$\begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix} + 10 = \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix} + \begin{bmatrix} 10 \\ 10 \\ 10 \end{bmatrix}$$

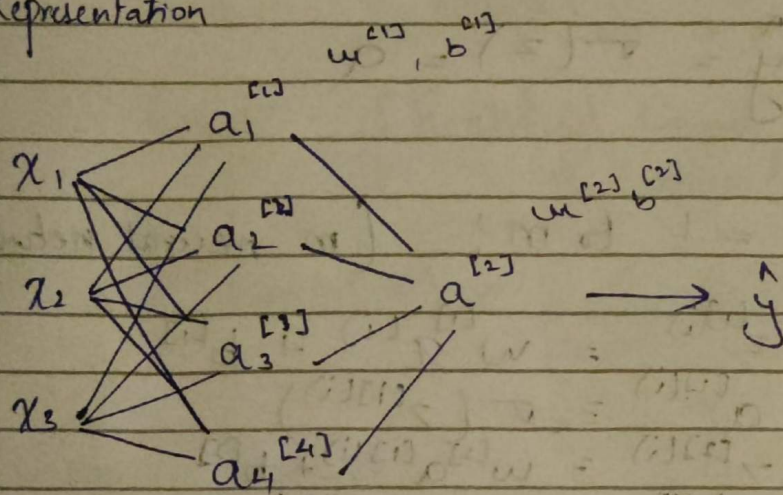
* Neural Networks





For multiple nodes just repeat this for each node and also gradient calc. in back propagation

Representation



$$z_1' = w_1'^T x + b_1', \quad a_1' = \sigma(z_1')$$

$$z_2 = w_2^T x + b_2, \quad a_2 = \sigma(z_2)$$

$$z_3 = w_3^T x + b_3, \quad a_3 = \sigma(z_3)$$

$$z_4 = w_4^T x + b_4, \quad a_4 = \sigma(z_4)$$

$$Z' = \begin{bmatrix} -w_1^T- \\ -w_2^T- \\ -w_3^T- \\ -w_4^T- \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

$$\therefore z' = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{bmatrix}$$

$$a' = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{bmatrix}$$

$$\hat{y} = \sigma(z) = a$$

for $i = 1$ to m : (m neural networks)

$$z^{[1]}(i) = w^{[1]} x^{(i)} + b^{[1]}$$

$$a^{[1]}(i) = \sigma(z^{[1]}(i))$$

$$z^{[2]}(i) = w^{[2]} a^{[1]}(i) + b^{[2]}$$

$$a^{[2]}(i) = \sigma(z^{[2]}(i))$$

$$X = \begin{bmatrix} | & | & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & | \end{bmatrix}$$

$$A^{[1]} = \begin{bmatrix} | & | & | \\ a^{[1]}(1) & a^{[1]}(2) & \dots & a^{[1]}(m) \\ | & | & | \end{bmatrix}$$

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$A^{[1]} = \sigma(z^{[1]})$$

$$z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma(z^{[2]})$$

② Activation function

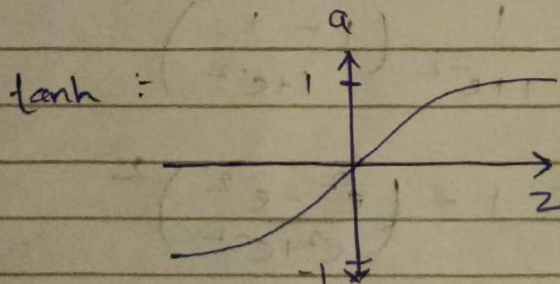
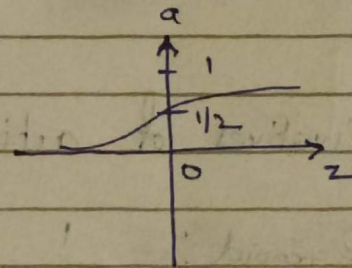
Eg: Sigmoid $[\sigma(z)]$ $a = \frac{1}{1+e^{-z}}$

Tanh $[\tanh(z)]$ $a = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

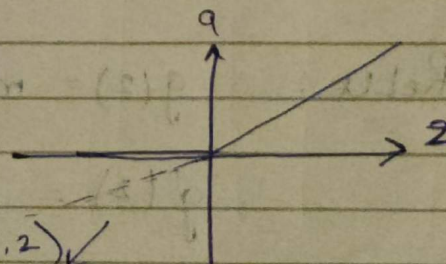
ReLU [Rectified Linear unit]
 $a = \max(0, z)$

$$a = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Graphs: Sigmoid :-



ReLU :-



$a = \max(0, 0.01, 2, 2)$ ✓
 Leaky ReLU

* We always use non-linear activation functions such as (ReLU, tanh, etc.) only for some places like regression or in output layer we can go ahead with linear functions.

$$\begin{aligned} \text{Ex: } z^{[1]} &= w^{[1]}x + b^{[1]} \\ a^{[1]} &= z^{[1]} \\ z^{[2]} &= w^{[2]}a^{[1]} + b^{[2]} \\ a^{[2]} &= z^{[2]} \end{aligned}$$

$$\begin{aligned} a^{[1]} &= z^{[1]} = w^{[1]}x + b^{[1]} \\ a^{[2]} &= z^{[2]} = w^{[2]}a^{[1]} + b^{[2]} \\ &= w^{[2]}(w^{[1]}x + b^{[1]}) + b^{[2]} \\ &= (w^{[2]}w^{[1]})x + w^{[2]}b^{[1]} + b^{[2]} \\ &= \underline{\underline{w'x + b'}} \end{aligned}$$

* Derivatives of activation functions

$$\text{Sigmoid: } \frac{1}{1+e^{-z}} \left(1 - \frac{1}{1+e^{-z}} \right)$$

$$\text{Tanh: } 1 - \left(\frac{e^z - e^{-z}}{e^z + e^{-z}} \right)^2$$

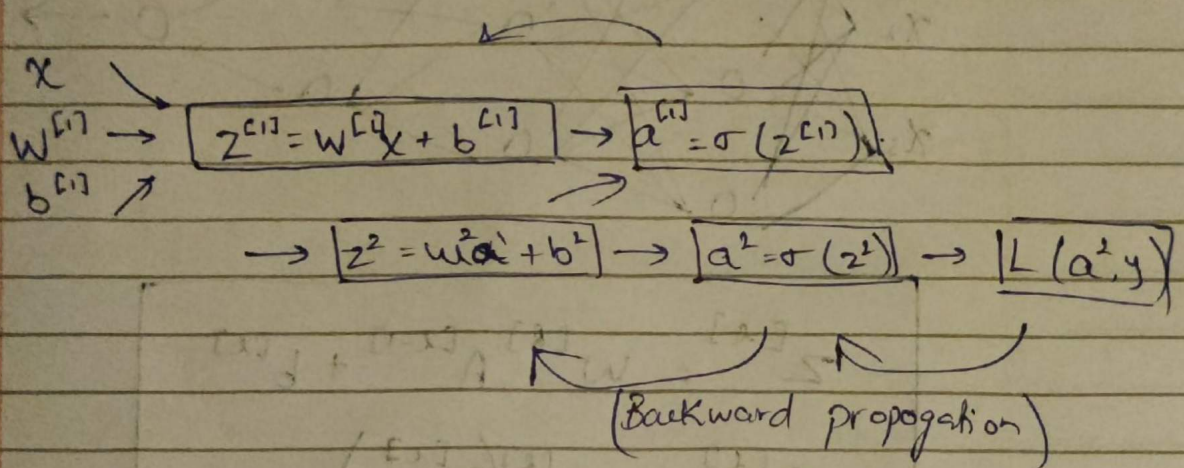
$$\text{ReLU: } g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 0, & \text{if } z < 0 \\ 1, & \text{if } z \geq 0 \end{cases}$$

Leaky ReLU : $g(z) = \max(0.01z, z)$

$$g'(z) = \begin{cases} 0.01, & \text{if } z < 0 \\ 1, & \text{if } z \geq 0 \end{cases}$$

+ Backward Propagation Formulas & derivations not written in detail



Summary

$$dz^{[2]} = a^{[2]} - y$$

$$dw^{[2]} = dz^{[2]} a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$dz^{[1]} = w^{[2]T} dz^{[2]} * g'(z^{[1]})$$

$$dw^{[1]} = dz^{[1]} x^T$$

$$db = dz^{[1]}$$

$$dz^{[2]} = A^{[2]} - y$$

$$dw^{[2]} = \frac{1}{m} dz^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} \text{np.sum}(dz^{[2]})$$

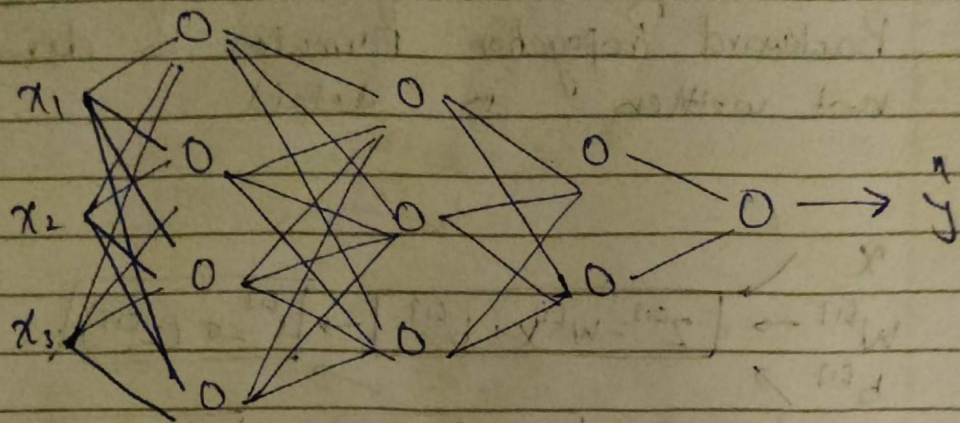
$$dz^{[1]} = w^{[2]T} dz^{[2]} * g'(z^{[1]})$$

$$dw^{[1]} = \frac{1}{m} dz^{[1]} x^T$$

~~Random Initialization~~

* Deep Neural Networks

- Basically NN with multiple hidden layers



$$z^{[L]} = W^{[L]} A^{[L-1]} + b^{[L]}$$
$$A^{[L]} = g^{[L]}(z^{[L]})$$

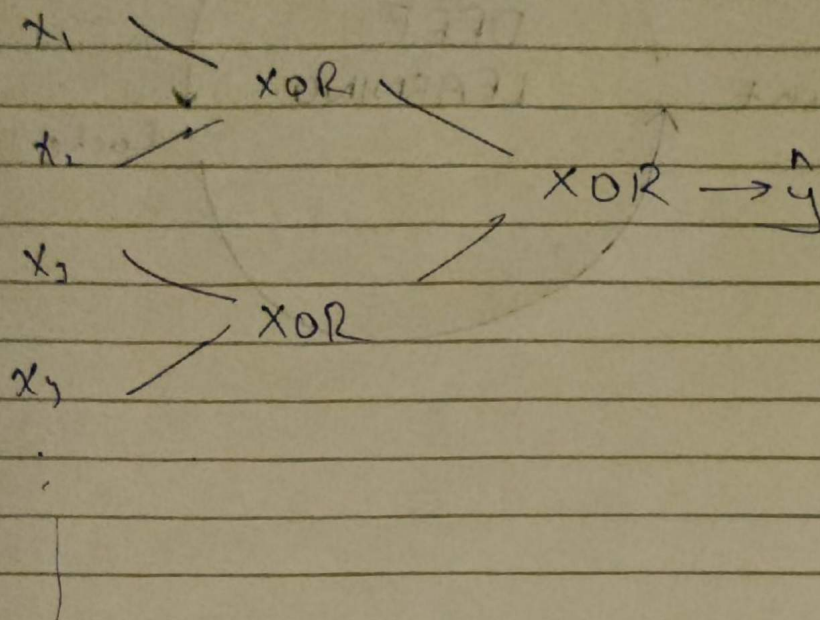
$$W^{[L]} = (n^{[L]}, n^{[L-1]})$$

$$b^{[L]} = (n^{[L]}, 1)$$

$$dW^{[L]} = (n^{[L]}, n^{[L-1]})$$

$$db^{[L]} = (n^{[L]}, 1)$$

* Circuit theory



$$\longleftrightarrow O(\log n)$$

$$y = x_1 \text{ XOR } x_2 \text{ XOR } x_3 \dots$$

Parameters & Hyperparameters

Parameters : $w^{[1]}$, $b^{[1]}$, $w^{[2]}$, $b^{[2]}$...

Hyperparameters :

- (i) Iterations
- (ii) Act. function
- (iii) Hidden layers
- (iv) Hidden units $n^{[1]}$, $n^{[2]}$

